City Infrastructure Project Requirements

Introduction:

This is a Requirements Specification document for a new web-based city infrastructure reporting platform. This system allows citizens to report problems in their city, such as potholes, broken streetlights, flooding, or damaged sidewalks. Users can submit reports with descriptions, photos, and locations. City staff can review these reports, assign them to the correct department, and track progress until the issue is resolved.

This document describes the scope, objectives and goal of the new system. In addition to describing non-functional requirements, this document models the functional requirements with use cases. This document is intended to direct the design and implementation of the target system in an object oriented language.

Project Name: City Infrastructure Reporting Document
Project Team: Ameen Ayyad, Anderson Varela Suarez, Brody Adams, Ellie Carroll, Ethan DeLaRosa, Roi Dupart, Jana El-Khatib, Javier Garcia Gomez, Makayla Hairston, Adin Hultin, Kelly Keith, Madeline Krehely, Hector Maes, Sophina Nichols, Carter Roberts, Ava Walker, Cameron Williams.

Project Scope:

"This project is a civic issue reporting and tracking system for the Greater New Orleans area that allows residents to submit non-social reports (with descriptions, photos, and locations) of city infrastructure problems and view progress toward resolution. City staff and management review, assign, and track these reports through completion, providing transparent status updates back to citizens"

<u>Basic Scope Points List:</u>

Who will be seeing this system:

- Citizens of the greater New Orleans Area (user)
- City Staff (user)
- Developers

Basic Use:

- People report issues to a service which city staff can then review and act upon.
- Citizens can view city progress to act upon certain issues.
- Citizens submit reports with descriptions and photos.
- City staff can review these reports, assign them to the correct department, and track progress until the issue is resolved.
- Not a social media app
- Updates users on the progress of their reports

System Purpose:

Users: City Staff, Citizens, Developers

Location: Greater New Orleans

System responsibilities:

- Database with a UI w/ Map interface
- Allow Different access to webpages based on account type/permissions
- Provide citizens with access to most up to date and accurate information from the city.
- Validate and Catalog Reports (multiple reports of the same thing should be grouped into one)

Need:

- Map Page for users to easily and conveniently view reports
    - Ex: https://www.511la.org/#:Alerts
    - Information Page describing the service
- Account sign up page to create accounts landing page
- Database Page where admin can view all reports
- Location tracking by distance from center point in the city (no more than 50 miles)
- Email Verification / Phone number pattern recognition (ex: 9 numbers)
    - This is a MOCKUP; this is not actually functional, we do not have an email or phone service; it should recognize the pattern of the input

Functional Objectives:

High Priority:

- Database coordinates correspond to map points when placing report area
    - GIS
- Allows citizens to submit infrastructure issue reports with a description and location
- Allows citizens to view the status of issues they submitted
- Allows city staff to review, accept and/or reject reports
- Allows city staff to view and update the status of an issue
- City Staff can assign acted-upon issues to a department and assign markers to them for "IN-PROGRESS", "HALTED/ON PAUSE" "REJECTED", "DONE", etc… This is used to keep reporters up-to-date on developments in their reported issues

Medium Priority:

- Citizens are able to upload photos with their issue reports
- Users are able to create accounts to track submitted issues
- Spam filter for reports to weed out repetitive garbage floods, improving readability for the city staff (removing all repeated items)

Low Priority:

- Boundary on map of just the new orleans area
- Drop-down selection menu when reporting for category of infrastructure damage (if "Other", a textbox for describing appears), which impacts next drop-down selection options for severity, since type of damage is relevant to metric of severity

Non-Functional Objectives:

Reliability:

- Relational Database to store reports
- Allow citizens to view a map of reports and issues in the city
- Real time updates upon page refresh
- System should be available to users (...)% of the time
- System ensures that issue status updates are consistently saved

Performance:

- Accessible from both a computer and phone browser
- Low wait times for uploads
- System supports multiple users at the same time without significant slowdown

Security:

- Reporting system immune to code injection, be it SQL or otherwise
- Password strength requirement for city staff/dev users, lowering chance of brute-force attacks succeeding
- User information is not shown to other users
- Confirmation emails are sent out on account creation

System Externals:

1. Taking the picture
2. Assigning the issue to a specific maintenance team
3. Actual repair
4. City staff user is a verified city employee
5. Citizen user is any user IN the greater New Orleans area.
6. Physical Verification of potholes

Use Cases:

Notes:

Use Case Name: **Register User**

Summary: Allow users to create an account by entering email, username, and password.

Basic Flow:

1. User indicates they want to register
2. The system requires the input of a username and a password
3. The system makes sure that the username inputted is not in use in accordance with the database, and that the password is strong enough to resist brute force attacks (at least 2 numbers and 1 special character)
4. If the username is not taken and the password is strong enough, the “User Preferences” use-case is executed.
5. The user is logged in under their new credentials and taken to the main map page.

Alternative Flow:

4→4.1. If either the username is already used or the password is not strong enough, credentials are not entered into the database and the user is not registered; simply takes the user back to the register page with an error message.

Extension Points:

→ User Preferences

Preconditions:

- N/A

Postconditions:

- The user can now perform all the functions (ex. reporting) exclusive to logged-in users, so long as they log in.

Business Rules:

- Registered user credentials are stored in the database only accessible to administrators and must not be easily extracted with code injection from any point in the system

Use Case Name: **Login User**

Summary: Allow previously registered users to login to their existing accounts.

Basic Flow:

1. User indicates they want to login
2. The system requests the username and password
3. The system verifies the username and password against all registered users in the database
4. The system accepts the credentials if no errors occur and sends the user to the main map page for logged-in users.

Alternative Flow:

4→4.1. If any errors occur or the user forgets their username/password, there is a recovery button under the login spaces; the recovery message is sent to the phone number or email address used to update users on their reports.

4→4.2. The system recognizes the credentials as those of an administrator and sends the user to the administrators' main page instead of the logged-in user main map page.

Extension Points:

→ N/A

Preconditions:

- The user is registered.

Postconditions:

- The user can now perform all the functions (ex. reporting) exclusive to logged-in users until they log out, which is as simple as the click of a button in the UI and the confirmation "YES" on a subsequent menu asking if they intend to log out.

Business Rules:

- Inputted password should appear hidden as asterisks per character unless a button is clicked to "show password" on the log-in menu

- Log-in page should be easily accessible from opening system, and should not take a very long time to load & process entered information, since reporting is contingent upon logging in and often happens out where issues are seen

Use Case Name: **User Preferences**
Summary: Allow users to edit and update their user settings and preferences in accordance with their account.
Basic Flow:

1. User indicates they want to change their account-specific settings or has just registered; primarily, the phone number or email address used to update them on their reports
2. A tab is opened within the website, with a list of features and settings to be adjusted. There is a button select between "Phone #" and "Email" and a subsequent text box which requires the entry of a phone number or email address to update which is used for updating users on reports.
3. The user may change these settings to their liking, with an option to apply changes once they are finished.
4. When the "apply" option is selected, the system checks that the email address or phone number entered is not already used by any users in the database, and also that if it is a phone number, it is a valid phone number, or that if it is an email address, that it is formatted like a real email.
5. If the phone number/email address is unused and valid, the user's report-update entry for the database is updated to the number/address.
6. The system applies these changes to the user's account, in accordance with their adjustments, and then takes them to the city map page.

Alternative Flow:

3→3.1. The user has an option to discard all changes at any time.
5→5.1. If the phone number/email address is already used or not in the valid format for either, nothing is saved and an error message appears on the refreshed same settings-change page.
6→6.1. An option is always available to the user to reset the settings they changed back to default, in the case that they change something and do not know how to change it back. In which case the system restores the user's preferences back to their defaults.

Preconditions:

- The user is either logged in and indicates wanting to change their preferences, or has just registered and must specify their preferences.

Postconditions:

- The user can now receive updates on their reports and any reports they may have specified wanting updates on from the city map, provided they have access to the phone number or email address they specified.

Business Rules:

- If work is expanded beyond semester, email address or phone number should receive verification code entered in user preferences page before it is registered under the user in the database; otherwise, too much work for scope of project

Use Case Name: **View City Map**
Summary: Allow both users and non-users to view a map of the New Orleans area.
Basic Flow:

1. The map encompassing the Greater New Orleans Area is the first thing seen on the webpage, alongside the login box in a minimized fashion, but can also be viewed in full size after logging in.
2. Red pins on the map correspond to reported potholes, flooding, etc. infrastructure
3. People can press on red pins to view a picture & description of the report, beginning the "View Report" use-case.
4. If the user is logged-in, they can indicate through a button on the full-size map page that they want to file a report, beginning the "File Report" use-case.

Extension Points:

→ View Report
→ File Report

Preconditions:

4. → User must be logged in.

Postconditions:

- The user can see all reports in the Greater New Orleans area under the system, and may even be able to file them if they are logged in.

Business Rules:

- Login is not necessary to view the city map, only to actually file reports and to get updates on them.

Use Case Name: **File Report**
Summary: Allow users, specifically New Orleans citizens, to file reports of infrastructure issues in certain areas.
Basic Flow:

1. User indicates, from a button listed with "File Report" on logged-in fullsize map page, that they want to report an infrastructure deficiency within the GNOA
2. The user must first select a place on the GNOA map page to place the pin for the report, since this is where the report will be shown for all users if the Admin reviews and approves the report.
3. The user is prompted to select, from a drop-down list, which category of infrastructure issue they are reporting; for example, whether it is a flooding issue, a pothole, a streetlight issue, or even "Misc./Other". From this step onward, there is always a "Back" button which takes the user to the previous step; see Business Rules for further details about implementation.
4. Once the user has selected the category, they are prompted to select, from a subsequently-dependent dropdown list, how severe the issue is– this is dependent on the category because categorization of issue often impacts heavily how severity levels of the issue are categorized; the basic blueprint is a level selection based on immediate urgency, which is the selection for "Misc./Other" as well

5. The user must type a title and description for the report, and must also provide the date (MM/DD/YYYY) that they noticed the infrastructure issue. They are able to, optionally, attach an image file to the report.
6. The user is shown a list of similar reports in the general area their pin is placed to ensure that they are not reporting a duplicate issue; they are allowed to expand the descriptions (and maybe images, if provided in those reports) for these reports, and at the end of their reviewing, select whether their report is original or not; if the report is original, the process continues.
7. Right before the report is complete and sent to the city to be reviewed (see "Review Report (Admin)" use-case, even though it isn't an extension point because this same end-user doesn't review the report right after/during this use-case), all the information the user has entered is shown to the user in the format of how a report is viewed by a user, and they are prompted one last time whether this form of the report is fine to submit to the city. If they press "Yes", their report is sent and complete. Otherwise, they are sent back to #6.

Alternative Flow:

5→5.1. If an image file is attached to the report and has metadata attached for date and/or location, it may automatically fill in the date for the report and update the location of the pin for the report. However, the user can, at any step of the report-filing process, re-place the pin or go back a few steps and replace the date without the image being technically re-entered when returning to the further steps, in case the metadata is wrong.

6→6.1. If the report is chosen as not original, the reporting process is terminated.

Extension Points:

N/A

Preconditions:

- The user is successfully registered & logged in.

Postconditions:

- The user's intended infrastructure issue report is now under consideration by the city and, if approved by them, may be shown on the map for any user to see and receive progress updates on.

Business Rules:

- After selecting the pin location for the first time, **the user should always be able to return to previous steps of the reporting process, without moving forward again refreshing the entered/replaced data for that forward-step**; For example, if the user manually enters a date for their report, then on the next step an image they attach updates the date with metadata that isn't accurate to when they first saw the issue, they should be allowed to return to the date entry point and fix the replacement without them later going forward to the image again refreshing the wrong date-replacement.

Use Case Name: **View Report (User)**

Summary: Allows both users and non-users to view and read-up on reports of infrastructure difficulties posted on the map of the New Orleans area.

Basic Flow:

1. User indicates they want to view a specific report by pressing on one of the red pins on the small, un-logged or enlarged, logged-in map.
2. The system loads the data associated with the report (i.e. what user made the report, the type of damage, how long ago it was made, any associated images, etc.) and displays it in a form-like popup. While the popup is open, the user cannot open any other reports in the same fashion.
3. User may view the progress of the city in acting upon the report, and if they are logged in, may choose to have updates of progress on the report sent to their specified phone number or email address whenever it is made by the city.
4. Whenever the user is done viewing the report, they can close the popup.

Extension Points:

→ N/A

Preconditions:

N/A

Postconditions:

- The user has viewed the report they desire to see based on the pin on the map.

Business Rules:

N/A

Use Case Name: **Review Report (Admin)**

Summary: Admin users with the city are able to look over individual reports. They are able to update the progress on each report as real life progress happens.

Basic Flow:

1. Admin user selects or requests to view a specific report.
2. The system verifies user authentication and confirms admin privileges.
3. The system retrieves the report data from the database.
4. System loads and displays the Admin View of the report (extends User View with additional administrative controls).
5. Admin reviews report details.
6. Admin modifies report progress and/or related administrative fields.
7. Admin selects “Save” or completes the session.
8. The system validates input data.
9. The system updates the report in the central database.
10. System confirms successful update and displays updated report state.

Alternative Flow:

2→2.1. If the user is not an admin, the system does not enable administrator view and simply takes the user to the regular map page for logged-in users.

3→3.1. If the requested report cannot be found, the system displays a “Report not found” message and redirects the admin to the report listing page.

3→3.2. If the system is unable to retrieve report data from the database, an error message is displayed and the failure is logged, then the admin is similarly redirected.

8→8.1. If validation fails during submission, the system highlights the invalid fields and prompts the admin to correct them before allowing the update.

9→9.1. If the database update fails, the system displays an error message, rolls back the transaction to prevent partial updates, and logs the failure.

Preconditions:

- Admin user is logged in and report exists within the database

Postconditions:

- The report the admin reviewed is updated accordingly & changes are logged.

Business Rules:

- Administrator approval should be necessary before a report filed is visible on the map page.
- Administrator user authentication methods should not be susceptible to code injection or brute-force attacks.

Use Case Name: **View Database (Admin)**

Summary: Admins look over all maintenance reports in a structured manner. This report should be a way to condense all of the collected data into a single report that can be easily viewed or printed.

Basic Flow:

1. The admin selects the option to download/view the database overview.
2. System verifies authentication and confirms admin privileges.
3. The system retrieves all maintenance report data from the database.
4. System aggregates and organizes the data into a structured summary format (.csv could be sufficient)
5. Admin may select an option to export or print the report.
6. The system confirms successful generation of the report.

Alternative Flow:

2→2.1. If the user is not an admin, the system denies access and redirects to the standard user map page.

3→3.1. If no maintenance reports exist, the system displays a message indicating that no data is available.

3→3.2. If the system fails to retrieve data from the database, an error message is displayed and the failure is logged before the admin is redirected to the main page.

6→6.1. If report generation or export fails, the system displays an error message and logs the issue without altering stored data.

Preconditions:

- Admin is logged in, maintenance reports exist within the database, and database is accessible through the web

Postconditions:

- The database report is accurate and reflective of the most recent version of the data since no data is modified.

Business Rules:

- Administrator user authentication methods should not be susceptible to code injection or brute-force attacks.
- Database should be as relational as possible

Situational Examples:

Someone is walking their dog down the street and there's a flooded section of sidewalk from a ground leak. The person can take a picture and look up our website to upload a report. They can monitor the repair over the next few days and know once it's been cleared so they can walk their dog there again. The realtime updates let them know beforehand if that's a good route to take or not.

Someone might like running at night but there is always a 3-block section in their route where the streetlights are all out. They can report this and have it be resolved so they can more safely engage in their routine activity.

There’s hundreds of pot holes in the city, but having this system would help track which potholes are the *most* inconvenient and so that could give the city an idea of where they should start, since filling them all would be a long process anyway. It would make the repair process more efficient and it would feel more responsive to what the people actually need.